

Como o senhor disse que poderia usar o Cython vou usar. gostei da forma que o Tiado ensinou.

Buid

```
~/Faculdade/Projeto De Blocos/Trabalhos/Projeto - Copia/TP2/Exercicio 1 git:(main) 26 • +699 -693 (1.361s)
python3 setup.py build_ext --inplace
running build_ext
Compiling exercicio.pyx because it changed.
[1/1] Cythonizing exercicio.pyx
building 'exercicio' extension
creating build
creating build/temp.linux-x86_64-cpython-312
x86_64-linux-gnu-gcc -fno-strict-overflow -Wsign-compare -DNDEBUG -g -O2 -Wall -fPIC -I/usr/include/python3.12 -c exercicio.c -o build/temp.linux-x86_64-cpython-312/exercicio.o -fopenmp
creating build/lib.linux-x86_64-cpython-312
x86_64-linux-gnu-gcc -shared -Wl,-O1 -Wl,-Bsymbolic-functions -Wl,-Bsymbolic-functions -Wl,-z,relro -g -fwrapv -O2 build/temp.linux-x86_64-cpython-312/exercicio.o -L/usr/lib/x86_64-linux-gnu -o build/lib.linux-x86_64-cpython-312/exercicio.cpython-312-x86_64-linux-gnu.so -fopenmp
copying build/lib.linux-x86_64-cpython-312/exercicio.cpython-312-x86_64-linux-gnu.so ->
```

threads

usei:

```
export OMP_NUM_THREADS=[QUANTIDADE DE THREADS]
python3 -c "import exercicio; exercicio.calcular_pi()"
```

```
~/Faculdade/Projeto De Blocos/Trabalhos/Projeto - Copia/TP2/Exercicio 1 git:(main) 31 • +8141 -693 (0.176s)
export OMP_NUM_THREADS=1
python3 -c "import exercicio; exercicio.calcular_pi()"

Iniciando cálculo com 1 thread(s).....
Pi calculado: 3.1415926535904264
Tempo de execução: 0.1491 segundos
```

```
~/Faculdade/Projeto De Blocos/Trabalhos/Projeto - Copia/TP2/Exercicio 1 git:(main) 31 • +8141 -693 (0.064s)
export OMP_NUM_THREADS=4
python3 -c "import exercicio; exercicio.calcular_pi()"

Iniciando cálculo com 4 thread(s).....
Pi calculado: 3.1415926535896825
Tempo de execução: 0.0375 segundos
```

```
~/Faculdade/Projeto De Blocos/Trabalhos/Projeto - Copia/TP2/Exercicio 1 git:(main) 31 • +8141 -693 (0.047s)
export OMP_NUM_THREADS=8
python3 -c "import exercicio; exercicio.calcular_pi()"

Iniciando cálculo com 8 thread(s).....
Pi calculado: 3.141592653589815
Tempo de execução: 0.0211 segundos
```

Como esperado o uso de paralelismo reduzi drasticamente o tempo de execução ao dividir a carga para multiplas threads e é uma grande prova do OpenMP na otimização ed calculo.